POLITECNICO
MILANO 1863

Prova finale – Progetto di reti logiche

Luca Lorusso (10831296), Gabriele Martorano (10822443)

1. Introduzione 1
1.1 Scopo del progetto 1
1.2 Specifiche 1
1.3 Interfaccia del componente 2
2. Design del componente 4
2.1 Macchina a stati 4
2.1.1 INIT 4
2.1.2 CONFIG 4
2.1.3 READ 4
2.1.4 FILTER 5
2.1.5 WRITE 5
2.1.6 DONE 5
2.1.7 WAIT START 5
2.2 State register 5
2.3 Scelte progettuali 5
3. Risultati sperimentali 7
3.1 Test benches 7
3.2 Report di sintesi 8
4. Conclusioni 11

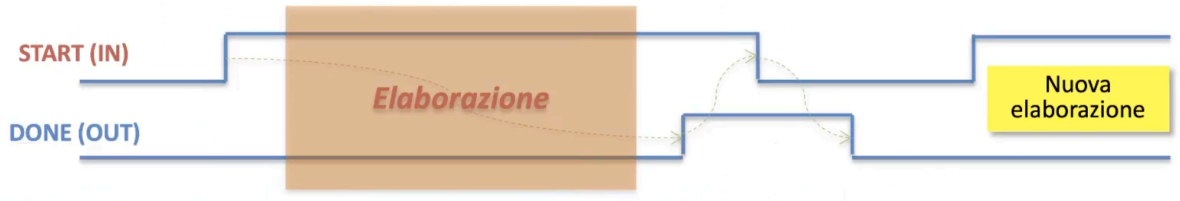
1. Introduzione

1.1 Scopo del progetto

Lo scopo del progetto è l'implementazione di un componente, in linguaggio VHDL, in grado di elaborare una sequenza di K valori campionati attraverso un filtro differenziale.
Questo filtro prende in ingresso una finestra mobile di 7 valori e restituisce un valore filtrato sulla base di una configurazione prelevata da memoria.

1.2 Specifiche

L'elaborazione inizia quando il test bench porta *i_start* alto, una volta terminata l'elaborazione il componente deve portare alto *o_done*, a questo punto *i_start* si abbassa (e di conseguenza il componente abbassa *o_done*) e il componente rimane in attesa di una nuova elaborazione. Il segnale di reset, asincrono, deve portare il componente allo stato di INIT e interrompere l'esecuzione.



I dati, ciascuno di dimensione 8 bit, sono memorizzati sequenzialmente in una memoria con indirizzamento al byte a partire da un indirizzo specificato (ADD).
A partire da tale indirizzo, i primi 17 byte sono impiegati per configurare il filtro differenziale, in particolare:

- i primi due byte (K1 e K2), concatenati, rappresentano la lunghezza K della sequenza dei dati da leggere. K1 è il byte più significativo;
- il terzo byte (S) indica, in base al valore del primo bit, se va applicato il filtro di ordine 3 o di ordine 5;
- dal quarto al decimo byte sono salvati i coefficienti del filtro di ordine 3, mentre dall'undicesimo al diciottesimo byte sono salvati i coefficienti del filtro di ordine 5.

Infine, dal diciottesimo byte (a partire da ADD), K byte vengono riservati alle K parole (W) sulle quali dev'essere applicato il filtro.
I valori filtrati (R) vanno scritti in memoria a partire dall'indirizzo successivo all'ultimo contenente i valori da leggere.
Il filtro ha la seguente funzione: $f'(i) = \frac{1}{n} \cdot \Sigma C_j \cdot f[i + j]$ con j che va da $-l$ e $+l$ e con l che vale 2, con $n \simeq 12$, per il filtro di ordine 3 e 3, con $n \simeq 60$, per il filtro di ordine 5.

Nel progetto, la divisione richiesta dal filtro differenziale non viene implementata tramite un'operazione di divisione vera e propria (costosa in hardware), bensì approssimata tramite una sequenza di shift destri (*shift_right*) sui valori.
Questo perché uno shift a destra di un numero binario equivale a una divisione per una potenza di due.
In particolare:

- Filtro ordine 3 → divisione per 12
- Filtro ordine 5 → divisione per 60

Queste divisioni sono approssimate così:

$\div 12 \rightarrow \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \frac{1}{1024} \rightarrow$ (x >> 4) + (x >> 6) + (x >> 8) + (x >> 10)

$\div 60 \rightarrow \frac{1}{64} + \frac{1}{1024} \rightarrow$ (x >> 6) + (x >> 10)

Tuttavia, quando il valore sum è negativo, è necessario correggere l'approssimazione introdotta dagli shift.
Lo shift a destra di un valore negativo in VHDL non è equivalente a una divisione matematica intera con approssimazione verso lo zero, ma tende ad arrotondare verso il basso, introducendo un errore sistematico di -1.
Se sum è negativo, dopo aver sommato i risultati degli shift deve essere aggiunto manualmente un valore positivo pari al numero di shift:

- + 4 se si sta dividendo per 12
- + 2 se si sta dividendo per 60

1.3 Interfaccia del componente

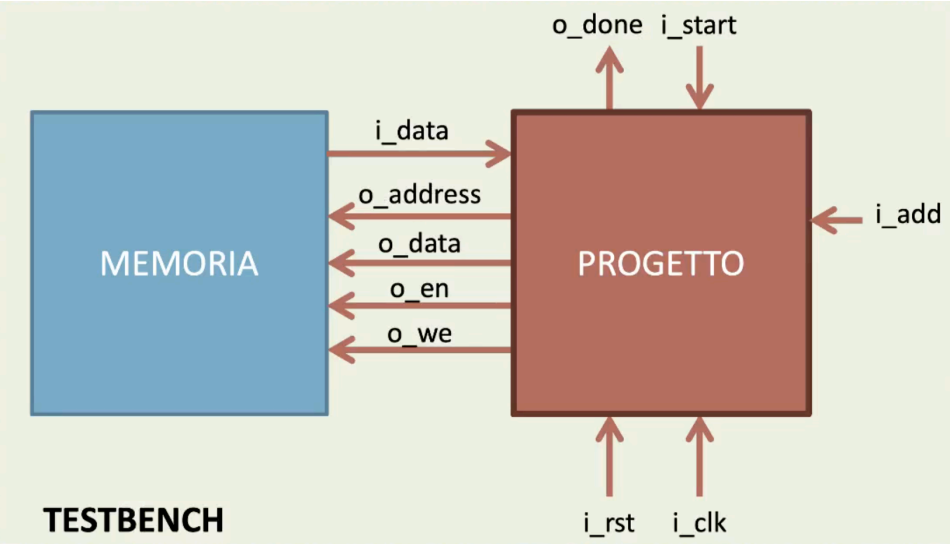
Il componente da descrivere ha un'interfaccia così definita:

```
entity project_reti_logiche is
    port (
        i_clk : in std_logic;
        i_rst : in std_logic;
        i_start : in std_logic;
        i_add : in std_logic_vector(15 downto 0);
        o_done : out std_logic;
        o_mem_addr : out std_logic_vector(15 downto 0);
        i_mem_data : in std_logic_vector(7 downto 0);
        o_mem_data : out std_logic_vector(7 downto 0);
        o_mem_we : out std_logic;
        o_mem_en : out std_logic
    );
end project_reti_logiche;
```

- *i_clk* è il segnale del clock, è generato dal test bench, ed è richiesto che il progetto funzioni con un valore di almeno 20 ns.

- *i_start* è il segnale di start, generato dal test bench, che il componente deve attendere abbia valore alto prima di poter iniziare un elaborazione.
- *i_rst* è il segnale di reset, usato per inizializzare il componente prima del segnale di start e per interrompere un'elaborazione e riportare il componente in attesa di una nuova elaborazione.
- *i_add* è il vettore contenente il primo indirizzo di memoria da cui iniziare a leggere i valori di configurazione ed è generato dal test bench.
- *o_done* è un segnale posto a valore alto dal componente quando l'elaborazione è terminata e va passato al test bench.
- *o_mem_addr* è il vettore contenente l'indirizzo di memoria, passato dal componente al test bench, per accedere in memoria
- *i_mem_data* è il vettore che viene letto dal componente dalla cella di memoria del test bench, definita da o_mem_addr.
- *o_mem_data* è il vettore che viene scritto dal componente nella memoria del test bench all'indirizzo o_mem_addr.
- *o_mem_en* è il segnale che, se alto, abilita l'accesso in memoria.
- *o_mem_we* è il segnale che, se alto, abilita la scrittura in memoria.

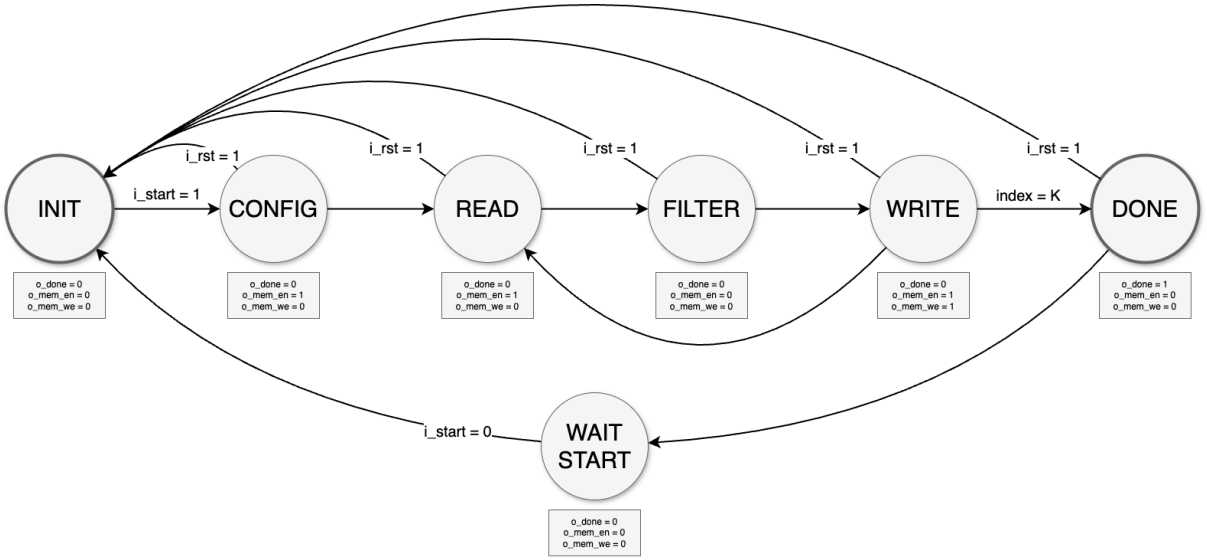
L'accesso del componente alla memoria del testbench è così gestita:



2. Design del componente

2.1 Macchina a stati

L'architettura del componente è basata su una macchina a stati finiti sincrona con il clock di sistema. La progettazione a stati permette di separare in modo ordinato le fasi di configurazione, acquisizione dati, elaborazione e scrittura dei risultati. Ogni stato corrisponde a una fase specifica dell'elaborazione, riducendo la complessità di controllo e migliorando la leggibilità e la gestibilità del progetto.



2.1.1 INIT

Stato iniziale della macchina, in cui si attende che *i_start* venga alzato e si inizializzano i valori per l'elaborazione. In caso venga alzato il segnale *i_rst*, in un qualsiasi momento della computazione, si torna in questo stato.

2.1.2 CONFIG

Stato in cui vengono letti i valori di configurazione del filtro. Viene abilitata la memoria ma non la scrittura, viene aggiornato il valore del vettore *o_mem_addr* e salvato il valore ricevuto dal vettore *i_mem_data*. La lettura viene fatta su cicli di clock distinti, gestiti dalla variabile step.

2.1.3 READ

Stato in cui vengono letti i valori della sequenza. Viene abilitata la memoria ma non la scrittura, viene aggiornato il valore del vettore *o_mem_addr* e salvato il valore ricevuto dal vettore *i_mem_data*. La finestra è aggiornata a ogni ciclo, ed è gestita tenendo conto dei bordi (i primi e ultimi elementi), applicando eventualmente riempimento con zeri (per gli estremi della sequenza). Infine viene disabilitata la

memoria, preparando il componente per FILTER. La lettura viene fatta su cicli di clock distinti, gestiti dalla variabile step.

2.1.4 FILTER

Stato in cui viene applicato il filtro sui 7 valori letti in READ in base al filtro configurato in CONFIG. Dunque viene calcolata la somma dei valori letti, moltiplicati per i rispettivi coefficienti, poi effettuata la divisione in base al filtro (utilizzando gli shift destri sul vettore binario, così da effettuare la corretta approssimazione della divisione) e infine, se i valori superano l'intervallo consentito, vengono riportati agli estremi. Step distingue i cicli di clock per la somma e per gli shift, evitando che i calcoli eccedano il tempo di clock.

2.1.5 WRITE

Stato in cui il valore elaborato viene scritto in memoria, portando alto *o_mem_we* e *o_mem_en* e aggiornando il valore del vettore *o_mem_addr*. Il valore di index viene incrementato, quando questo è uguale a K lo stato successivo è DONE, altrimenti si torna in READ per l'elaborazione della successiva finestra di valori. Step distingue i cicli di clock in cui avviene l'aggiornamento dell'indirizzo di memoria e l'effettivo salvataggio del valore di scrittura in essa.

2.1.6 DONE

Stato finale dell'elaborazione, *o_done* viene alzato e vengono abbassati i segnali relativi alla memoria.

2.1.7 WAIT START

Stato in cui si attende che, dopo la fine dell'elaborazione, *i_start* venga portato a 0 per tornare in INIT e iniziare una nuova elaborazione.

2.2 State register

Il registro di stato controlla l'evoluzione della macchina a stati finiti in modo sincrono con il clock e asincrono con il reset. Al reset, il sistema si porta nello stato INIT e azzera il segnale *o_done*. A ogni ciclo di clock, lo stato corrente viene aggiornato con il prossimo stato calcolato. Quando si raggiunge lo stato DONE, *o_done* viene attivato per segnalare la fine del processo. Questo meccanismo garantisce un controllo ordinato del flusso operativo del sistema.

2.3 Scelte progettuali

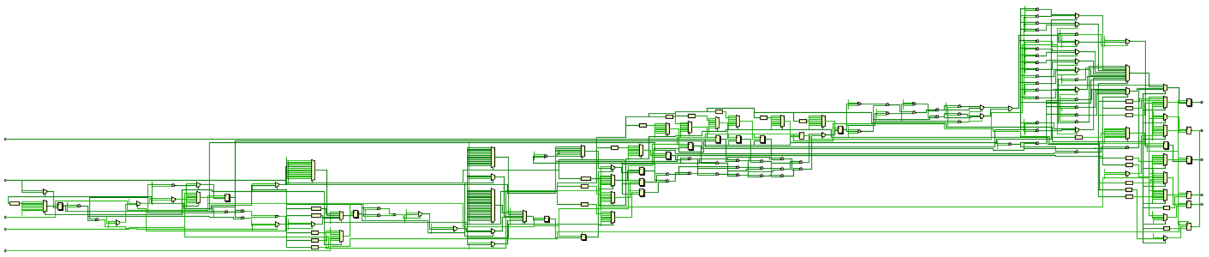
Segnali interni:

- *current_state* – Segnale, tipo state_type
  Stato corrente della macchina a stati finiti.
- *next_state* – Segnale, tipo state_type
  Prossimo stato da assumere nella macchina a stati.

- *step* – Segnale, tipo integer
  Variabile di controllo per scansionare i cicli all'interno di ciascuno stato.
- *index* – Variabile, tipo integer
  Indice corrente della sequenza da leggere e filtrare.
- *k* – Variabile, tipo std_logic_vector(15 downto 0)
  Valore K letto da memoria, per la lunghezza della sequenza da elaborare.
- *s* – Variabile, tipo std_logic
  Bit che determina quale dei due insiemi di coefficienti usare (filtro ordine 3 o 5).
- *c0 - c6* – Variabili, tipo std_logic_vector(7 downto 0)
  Coefficienti del filtro letti dalla memoria, utilizzati nel calcolo del valore filtrato.
- *w* – Variabile, tipo std_logic_vector(55 downto 0)
  Finestra mobile di 7 valori, su cui viene applicato il filtro.
- *sum* – Variabile, tipo std_logic_vector(17 downto 0)
  Somma pesata intermedia tra coefficienti e valori della finestra.
- *r* – Variabile, tipo std_logic_vector(17 downto 0)
  Risultato finale dell'elaborazione del filtro, prima della saturazione.

Nel progetto si è scelto di utilizzare variabili locali per memorizzare i dati di configurazione e i valori letti, così da limitare la latenza delle assegnazioni, migliorando l'efficienza del codice. Le variabili vengono aggiornate immediatamente all'interno del ciclo di clock, evitando la necessità di segnali temporanei.
Per rappresentare la finestra mobile di 7 valori, è stato impiegato un unico vettore w di 56 bit, suddiviso in 7 sottosequenze da 8 bit ciascuna. Questo consente una gestione compatta e ordinata della finestra, con accesso diretto agli elementi tramite slicing (*w(7 downto 0), w(15 downto 8),* ecc.) e aggiornamento tramite shift sinistro. La scelta riduce la complessità del codice e facilita l'applicazione del filtro differenziale.

A seguire lo schematic del componente pre sintesi:



3. Risultati sperimentali

3.1 Test benches

Per verificare il corretto comportamento del componente e assicurare la copertura di percorsi più ampia possibile abbiamo definito, oltre ad alcuni test bench con sequenze di parole, configurazioni e dimensioni casuali, i seguenti test:

- **Corretta elaborazione del filtro di entrambi gli ordini**
  Sono stati verificati entrambi i comportamenti del filtro differenziale, corrispondenti ai due possibili valori del parametro S (0 e 1), i quali determinano la porzione di coefficiente da utilizzare. Nel caso S = 0, vengono impiegati i primi 7 coefficienti, mentre con S = 1 vengono utilizzati i secondi 7. Per ciascun caso, sono state generate sequenze di input con valori noti e confrontati i risultati ottenuti con i valori attesi. I test hanno dimostrato che l'algoritmo seleziona correttamente i coefficienti e applica il filtro secondo la configurazione.

  ESEMPIO
  - Numero valori: 24
  - Ordine filtro: 3
  - Coefficienti: 0, -1, 8, 0, -8, 1, 0
  - Sequenza in lettura: 32, -24, -35, 0, 46, -54, -39, -22, -53, -47, 12, 11, 11, …
  - Sequenza in scrittura: 11, 43, -13, -54, 33, 53, -28, 8, 18, -38, -31, 7, -24, …

- **Lunghezza minima della sequenza**
  È stato verificato il corretto funzionamento del filtro differenziale, sia per filtro di ordine 3 che di ordine 5, anche nel caso limite in cui la sequenza da elaborare contiene solo 7 elementi (K = 7), cioè il numero minimo di valori richiesto affinché il filtro possa applicare integralmente i 7 coefficienti previsti.

- **Lunghezza massima della sequenza**
  La memoria indirizzabile nel sistema è composta da 2^{16} = 65.536 byte, essendo gli indirizzi a 16 bit. Di questi, i primi 17 byte sono riservati alla configurazione del filtro, che include i parametri K1, K2, S e i 14 coefficienti C0 - C6, distinti per ciascun tipo di filtro.
  Per ogni elemento della sequenza da elaborare, è necessario:
  - 1 byte per la lettura del valore originale;
  - 1 byte per la scrittura del risultato elaborato.

  Se K rappresenta la lunghezza della sequenza, lo spazio complessivamente occupato in memoria sarà quindi:
  17 (configurazione) + K (lettura) + K (scrittura) = 17 + 2K
  Affinché l'indirizzamento non ecceda il limite massimo della memoria, si impone la seguente condizione:
  $17 + 2K \leq 65.536 \Rightarrow K \leq 32.759$ (risultato arrotondato per difetto)

Pertanto, il componente dev'essere in grado di gestire una sequenza di massimo 32.759 elementi, rispettando i vincoli di memoria disponibili; il nostro componente è stato testato su tale K e garantisce una corretta elaborazione.

- **Valori di R superiori al limite definito dall'intervallo**
  Sono state generate sequenze con coefficienti e valori estremi in modo tale che la somma filtrata R non risultasse superiore al massimo rappresentabile (127) o inferiore al minimo (-128). I risultati mostrano che il componente applica correttamente la saturazione di R a 127 o -128 quando necessario, come previsto dalla specifica. Questo comportamento è fondamentale per evitare overflow e garantire la correttezza del risultato in uscita. In particolare abbiamo anche esteso la lunghezza di sum e r da 16 a 18 bit per poter eseguire anche il caso di calcolo peggiore.

  ESEMPIO
  - Coefficienti: 127, 127, 127, 127, 127, 127, 127
  - Sequenza in lettura: -128, -128, -128, -128, -128, -128, -128

  $[\ 127 \cdot (\ -128\ )\ ] \cdot 7 = -113.792$
  $2^{16} = 65.536 < |\ -113.792\ | < 2^{17} = 131.072$
  Il 18° bit serve per la gestione di eventuali overflow e mantenimento del segno

- **Molteplici elaborazioni e reset**
  È stata simulata una sequenza di avvii (*i_start*) e reset (*i_rst*) multipli in momenti diversi, per verificare che il componente possa ripartire da zero senza conservare residui di stato tra una sessione di elaborazione e la successiva. Il sistema risponde correttamente al reset asincrono, ripristinando lo stato iniziale e consentendo una nuova elaborazione indipendente dalle precedenti. Anche in presenza di elaborazioni consecutive, la gestione degli indici e dei segnali di controllo è risultata corretta.

3.2 Report di sintesi

Il componente è risultato sintetizzabile correttamente senza errori o warning. Il progetto sintetizzato mostra il seguente utilizzo delle risorse logiche, ampiamente sotto i valori limite e non introduce alcuna criticità:

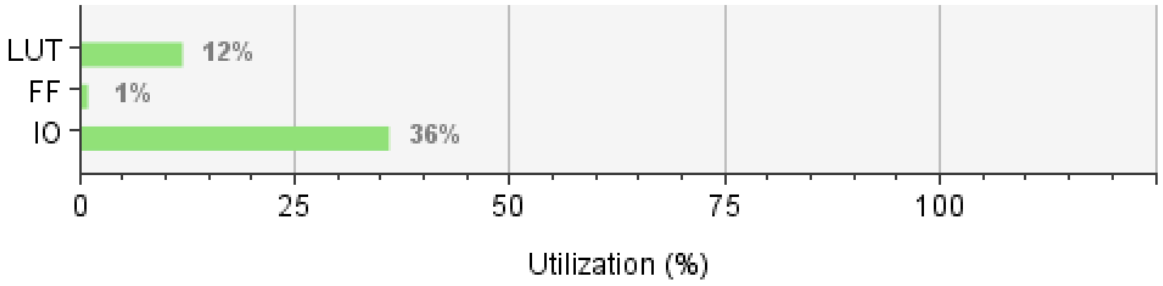
- LUT: 936 su 8000 (11,70%)
- FF: 222 su 16000 (1,39%)
- IO: 54 su 150 (36,00%)

In particolare:
- *LUT* (Look-Up Table): misura la logica combinatoria implementata
- *FF* (Flip-Flop): indicano i registri usati per memorizzare dati
- *IO* (Input/Output): indicano i pin esterni occupati dal progetto

Inoltre non vengono inferiti latch, il che indica che il progetto non contiene elementi di memoria generati in modo non intenzionale durante la sintesi.
I latch, infatti, sono circuiti sequenziali non sincronizzati con il clock, che possono nascere per errore se, ad esempio, un if in VHDL non assegna sempre un valore a una variabile o a un segnale in tutti i percorsi possibili. Questo è pericoloso perché i latch si comportano in modo meno prevedibile dei flip-flop e possono causare problemi di timing e errori di funzionamento.

Quindi, il fatto che non ci siano latch conferma che:
- Tutti i segnali interni sono correttamente aggiornati a ogni ciclo di clock.
- Il comportamento del circuito è deterministico.
- La macchina a stati e i processi sono completamente controllati dal clock.

Resource	Utilization	Available	Utilization %
LUT	936	8000	11.70
FF	222	16000	1.39
IO	54	150	36.00



Dal punto di vista temporale, il circuito rispetta tutti i vincoli di timing:
- Worst Negative Slack (WNS): 7,058 ns
- Worst Hold Slack (WHS): 0,150 ns
- Worst Pulse Width Slack (WPWS): 9,500 ns
- Failing Endpoints: 0 per tutte le analisi

In particolare:
- *Worst Negative Slack* (WNS): margine minimo sui vincoli di setup. Un valore positivo indica che tutti i segnali arrivano in tempo prima del fronte di clock.
- *Worst Hold Slack* (WHS): margine minimo sui vincoli di hold. Un valore positivo garantisce la stabilità dei dati subito dopo il fronte di clock.
- *Worst Pulse Width Slack* (WPWS): verifica che gli impulsi del clock rispettino la durata minima richiesta. Margine positivo assicura un clock stabile.
- *Failing Endpoints*: numero di percorsi che violano i vincoli temporali. Un valore pari a 0 certifica il rispetto completo di setup, hold e clock.

Il valore positivo del Worst Negative Slack indica che il delay del percorso critico è pari a 12,942 ns (20 ns - 7,058 ns), ben al di sotto del periodo di clock. Il circuito opera con ampio margine rispetto al limite imposto dal clock ed è privo di violazioni sia di setup che di hold. Il progetto è quindi stabile e conforme alle specifiche.

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	7,058 ns	Worst Hold Slack (WHS):	0,150 ns	Worst Pulse Width Slack (WPWS):	9,500 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	445	Total Number of Endpoints:	445	Total Number of Endpoints:	223

All user specified timing constraints are met.

4. Conclusioni

Il sistema progettato soddisfa pienamente tutti gli obiettivi richiesti, rispettando le specifiche funzionali e temporali. Il componente realizzato ha dimostrato di operare correttamente sia in simulazione pre-sintesi che in ambiente post-sintesi, superando con successo tutti i test eseguiti, sia quelli forniti dal docente che quelli autogenerati per verificare le condizioni limite, al fine di garantire la copertura più ampia possibile.
Come evidenziato nella sezione di sintesi, il progetto rispetta i vincoli temporali richiesti, in particolare il periodo minimo di clock pari a 20 ns, con un margine di sicurezza ampio e assenza di criticità sui percorsi temporali, evitando inoltre elementi indesiderati quali latch o percorsi combinatori non ottimizzati.
Alla luce dei risultati ottenuti, si ritiene che il componente descritto sia conforme alle specifiche assegnate, robusto e affidabile nel comportamento.